

Pandas Library –

1. What is Pandas?

Pandas is a Python library used for:

- Data Analysis
- Data Cleaning
- Data Manipulation
- Data Visualization Preparation
- Working with CSV, Excel, SQL, JSON files

It is built on top of **NumPy** and is one of the most important libraries in Data Science, Machine Learning, and AI.

Installation

```
pip install pandas
```

Import

```
import pandas as pd
```

2. Pandas Data Structures

Pandas has two main data structures:

A) Series (1D)

A Series is like a single column.

```
import pandas as pd
```

```
s = pd.Series([10, 20, 30, 40])  
print(s)
```

Output:

```
0    10  
1    20  
2    30
```

```
3 40
dtype: int64
```

B) DataFrame (2D)

A DataFrame is like an Excel table.

```
data = {
    "Name": ["Harshil", "Raj"],
    "Age": [21, 22]
}
```

```
df = pd.DataFrame(data)
print(df)
```

Output:

```
   Name  Age
0 Harshil  21
1    Raj   22
```

3. Creating DataFrames

From Dictionary

```
df = pd.DataFrame({
    "Name": ["A", "B", "C"],
    "Marks": [80, 90, 70]
})
```

From List

```
data = [
    ["A", 80],
    ["B", 90]
]
```

```
df = pd.DataFrame(data, columns=["Name", "Marks"])
```

From NumPy Array

```
import numpy as np

arr = np.array([[1,2],[3,4]])

df = pd.DataFrame(arr)
```

4. Reading Files

CSV File

```
df = pd.read_csv("students.csv")
```

Excel File

```
df = pd.read_excel("students.xlsx")
```

JSON File

```
df = pd.read_json("data.json")
```

5. Saving Files

CSV

```
df.to_csv("output.csv")
```

Excel

```
df.to_excel("output.xlsx")
```

JSON

```
df.to_json("output.json")
```

6. Viewing Data

First Rows

`df.head()`

Last Rows

`df.tail()`

Shape

`df.shape`

Output:

(rows, columns)

Columns

`df.columns`

Data Types

`df.dtypes`

Information

`df.info()`

Statistics

`df.describe()`

7. Selecting Data

Single Column

`df["Name"]`

Multiple Columns

`df[["Name", "Marks"]]`

Row by Index

```
df.loc[0]
```

Row by Position

```
df.iloc[0]
```

8. Filtering Data

Marks Greater Than 80

```
df[df["Marks"] > 80]
```

Multiple Conditions

```
df[(df["Marks"] > 80) & (df["Age"] > 18)]
```

OR Condition

```
df[(df["Marks"] > 80) | (df["Age"] > 18)]
```

9. Adding Columns

```
df["Grade"] = ["A", "B", "A"]
```

Calculated Column

```
df["Bonus"] = df["Marks"] + 5
```

10. Deleting Columns

```
df.drop("Grade", axis=1)
```

Delete Permanently

```
df.drop("Grade", axis=1, inplace=True)
```

11. Renaming Columns

```
df.rename(columns={"Marks":"Score"})
```

12. Handling Missing Values

Check Missing Values

```
df.isnull()
```

Count Missing Values

```
df.isnull().sum()
```

Remove Missing Values

```
df.dropna()
```

Fill Missing Values

```
df.fillna(0)
```

Fill With Mean

```
df["Marks"].fillna(df["Marks"].mean())
```

13. Sorting Data

Ascending

```
df.sort_values("Marks")
```

Descending

```
df.sort_values("Marks", ascending=False)
```

14. Aggregation Functions

Sum

```
df["Marks"].sum()
```

Mean

```
df["Marks"].mean()
```

Max

```
df["Marks"].max()
```

Min

```
df["Marks"].min()
```

Count

```
df["Marks"].count()
```

15. GroupBy

Example

```
df.groupby("Department")["Salary"].mean()
```

Output:

```
Department
HR      30000
IT      50000
Sales   40000
```

16. Merge DataFrames

Inner Join

```
pd.merge(df1, df2, on="ID")
```

Left Join

```
pd.merge(df1, df2, on="ID", how="left")
```

Right Join

```
pd.merge(df1, df2, on="ID", how="right")
```

Outer Join

```
pd.merge(df1, df2, on="ID", how="outer")
```

17. Concatenation

Row Wise

```
pd.concat([df1, df2])
```

Column Wise

```
pd.concat([df1, df2], axis=1)
```

18. String Operations

Uppercase

```
df["Name"].str.upper()
```

Lowercase

```
df["Name"].str.lower()
```


Length

```
df["Name"].str.len()
```

Replace

```
df["Name"].str.replace("A","X")
```

19. Date and Time

```
df["Date"] = pd.to_datetime(df["Date"])
```

Year

```
df["Date"].dt.year
```

Month

```
df["Date"].dt.month
```

Day

```
df["Date"].dt.day
```

20. Apply Function

```
df["Marks"] = df["Marks"].apply(lambda x: x+5)
```

21. Pivot Table

```
pd.pivot_table(  
    df,  
    values="Sales",  
    index="Region",  
    aggfunc="sum"  
)
```

22. Crosstab

```
pd.crosstab(df["Gender"], df["Result"])
```

23. Duplicate Data

Check Duplicates

```
df.duplicated()
```

Remove Duplicates

```
df.drop_duplicates()
```

24. Indexing

Set Index

```
df.set_index("Name")
```

Reset Index

```
df.reset_index()
```

25. MultiIndex

```
df.set_index(["Department", "Name"])
```

26. Window Functions

Rolling Mean

```
df["Sales"].rolling(3).mean()
```

Expanding Sum

```
df["Sales"].expanding().sum()
```

27. Advanced Operations

Query

```
df.query("Marks > 80")
```

Map

```
df["Grade"] = df["Marks"].map({
    90:"A",
    80:"B",
    70:"C"
})
```

Value Counts

```
df["Grade"].value_counts()
```

28. Performance Tips

Use Vectorized Operations

Good:

```
df["Marks"] + 5
```

Bad:

```
for i in df["Marks"]:
    print(i+5)
```

29. Pandas With Matplotlib

```
import matplotlib.pyplot as plt
```

```
df["Marks"].plot()  
plt.show()
```

30. Real-World Data Analysis Workflow

Step 1

Load Data

```
df = pd.read_csv("data.csv")
```

Step 2

Inspect Data

```
df.head()  
df.info()
```

Step 3

Clean Data

```
df.dropna()  
df.drop_duplicates()
```

Step 4

Analyze

```
df.groupby("Department")["Salary"].mean()
```

Step 5

Visualize

```
df.plot()
```

Most Important Pandas Functions for Interviews

pd.read_csv()
pd.DataFrame()
pd.Series()
head()
tail()
info()
describe()
loc[]
iloc[]
groupby()
merge()
concat()
pivot_table()
sort_values()
dropna()
fillna()
apply()
map()
value_counts()
query()
drop_duplicates()
set_index()
reset_index()

Pandas Learning Roadmap

1. Series
2. DataFrame
3. Read/Write Files
4. Data Selection
5. Filtering
6. Missing Values
7. Sorting
8. Aggregation
9. GroupBy
10. Merge & Join
11. String Operations
12. DateTime

- 13. Pivot Tables
- 14. Advanced Analysis
- 15. Data Visualization
- 16. Real Projects